

Sampson's Monks

In this project, you will analyze and model Sampson's Monks dataset, a classic dataset in social network analysis.

By completing this project, you will:

- Work with matrices and lists in applied settings
- Practice row/column operations for network statistics
- Use functions, control flow, and libraries
- Understand the contrast between random vs. real-world networks

You will submit:

- A .R file with your code (well commented)
- A README.md describing your process, results, and reflections

Both files should be pushed to our GitHub Classroom by the due date (October 1 at 11:59PM).

Part 0 - Setting Up

Open your R app and install and load the necessary packages for this project: `lda`, `igraph`, and `networkD3`. `lda` contains the monks data.

Part 1 - Visualizing the Network

Create two network visualizations about monk relationships using one of the matrices in the "sampson" dataset from the `lda` package, such as `SAMPLK2`:

1. A static plot using `igraph::plot.igraph()`
2. An interactive plot using `networkD3::forceNetwork()`

Before plotting, check what data structure your plotting function accepts: `igraph` accepts an adjacency matrix. `networkD3` needs dataframes of nodes and edges, not an adjacency matrix. You will need to write code to transform the matrix into that format.

Part 2 - Summary Statistics on a Sociomatrix

Now that we have our sociomatrix, we can find summary statistics on it to better understand our data. Choose one sociomatrix (other than `SAMPLK1`, the example used in class) from the dataset. **Compute summary measures:**

1. Out-degree
2. In-degree
3. Mean tie strength

4. Store results in a list that keeps track of different measures.
5. Make a barplot of the in-degree statistic.
6. Which monk is most liked? Which monk is least liked?

Part 3 - A simple model of a social network

One way to learn more about the social dynamics underlying the monk's network is to compare it to a network generated using a simple but well-understood mechanism; that is to say: a model.

Write a function that will generate a network according to the rule that each monk assigns their likes (or dislikes) completely at random. This function should be general enough to allow the user to generate a network of any size. (*Hint: for loops can be helpful here*)

Feel free to add additional complexity into your model (for example, assigning the likes randomly but with unequal probabilities).

Part 4 - Comparison: Model vs Observations

Apply the same techniques you used in parts 1 and 2 (visualization and summary statistics) to **describe the structure of the network that was simulated by your model**. In 3-5 sentences, **compare these networks**: In what ways is the observed network `monk_mat` similar to your generated network? In what ways is it different? What patterns appear in the observed network that are missing from your generated one?

Submission

On GitHub Classroom, submit:

- `project.R` — your R code with comments explaining Parts 1, 2, 3
- `README.md` — with your answers to Part 4